

Problem Name: **Print Digit with Recursion (49)**

Summary of Problem:

There will be given a string and we have to print the digit separately using recursion.

Code:

```
#include<bits/stdc++.h>
using namespace std;
#define ll long long int
#define str string
#define newl cout<<endl
void print_digit(int n,str p,int x)
{
    if(x==n)
    {
        return;
    }
    cout<<p[x]<<" ";
    x++;
    print_digit(n,p,x);
}
int main()
{
    int n;
    cin>>n;
    while(n-->0)
    {
        str r;
        cin>>r;
        int m=r.size();
        print_digit(m,r,0);
        newl;
    }
}
```

Output:

Input	Output
3	1 2 1
121	3 9
39	1 2 3 4 5 6
123456	

Screenshot of Accepted Bar:

224151545	Practice: Arsenic33	223339D - 15	GNU C++20 (64)	Accepted	0 ms	0 KB	2023-09-20 13:57:08	2023-09-20 13:57:08
-----------	------------------------	---------------------------------	----------------------	----------	------	------	------------------------	------------------------

Comments of mine:

Here I have used 'print_digit' recursive function to call the method recursively and every time I have increased the index value of it.

Problem Name: **Factorial (50)**

Summary of Problem:

There will be given an integer and we have to print the factorial value of it by recursion.

Code:

```
#include<bits/stdc++.h>
using namespace std;
#define ll long long int
#define str string
#define newl cout<<endl
ll f(ll x)
{
    if(x==1)
    {
        return 1;
    }
    return x*f(x-1);
}
int main()
{
    ll n;
    cin>>n;
    cout<<f(n);
}
```

Output:

Input	Output
5	120

Screenshot of Accepted Bar:

224151963	Practice: Arsenic33	2233391 - 9	GNU C++20 (64)	Accepted	0 ms	0 KB	2023-09-20 14:00:49	2023-09-20 14:00:49
-----------	------------------------	-----------------------------	----------------------	----------	------	------	------------------------	------------------------

Comments of mine:

Here I have used 'f' recursive function which calls itself by decreasing the value and when hit the base case it return the value to the recursion.

Problem Name: **Combination (51)**

Summary of Problem:

There will be given the value of n and r and we have to calculate nC_r value of it.

Code:

```
#include<bits/stdc++.h>
using namespace std;
#define ll long long int
#define str string
#define newl cout<<endl
double ff(double i)
{
    if(i<=1)
    {
        return 1;
    }
}
```

```

    return i*ff(i-1);
}
int main()
{
    int n,r;
    cin>>n>>r;
    if(n==r)
    {
        cout<<1;
        return 0;
    }
    double x=ff(n),y=ff(r),z=ff(n-r);
    cout<<fixed<<setprecision(0)<<(x/(y*z));
}

```

Output:

Input	Output
4 2	6
3 3	1

Screenshot of Accepted Bar:

226878738	Practice: Arsenic33	2233391 - 17	GNU C++20 (64)	Accepted	15 ms	0 KB	2023-10-06 16:15:37	2023-10-06 16:15:37
-----------	------------------------	---------------------------------	----------------------	----------	-------	------	------------------------	------------------------

Comments of mine:

Here I have used 'double' data type which can calculate the factorial value of long range and used recursion to solve the problem.

Problem Name: **Knapsack (52)**

Summary of Problem:

There will be given 2 integers which are the number of items and the max capacity of knapsack. We have to print the maximum value that can get from the items.

Code:

```

#include<bits/stdc++.h>
using namespace std;
#define ll long long int
#define str string
#define newl cout<<endl
const int N=105;
const int M=1e5+5;
ll dp[N][M],wt[N],value[M];
ll Knapsack(int n,int cap)
{
    if(n==0)
    {
        return 0;
    }
    if(dp[n][cap]!=-1)
    {
        return dp[n][cap];
    }
    if(cap<wt[n])
    {
        ll ans=Knapsack(n-1,cap);
    }
}

```

```

        dp[n][cap]=ans;
        return ans;
    }
    ll ans1=value[n]+Knapsack(n-1, cap-wt[n]);
    ll ans2=Knapsack(n-1, cap);
    ll ans=max(ans1, ans2);
    dp[n][cap]=ans;
    return ans;
}
int main()
{
    int n, cap;
    cin >> n >> cap;
    memset(dp, -1, sizeof(dp));
    for(int i=1; i<=n; i++)
    {
        cin >> wt[i] >> value[i];
    }
    cout << Knapsack(n, cap) << endl;
}

```

Output:

Input	Output
3 8 3 30 4 50 5 60	90

Screenshot of Accepted Bar:

224695781	Practice: Arsenic33	223339U - 14	GNU C++20 (64)	Accepted	62 ms	82956 KB	2023-09-23 08:10:17	2023-09-23 08:10:17
-----------	------------------------	---------------------------------	----------------------	----------	-------	----------	------------------------	------------------------

Comments of mine:

Here I have used dynamic programming to solve this problem where everything is calculated by recursively and 'dp' variable helps to minimize the step by storing the sub problem solution.

Problem Name: **Creating Expression (53)**

Summary of Problem:

There will be given n integers and a value. We have to determine if it can possible to get the value from the integers by putting + or – sign before them.

Code:

```

#include<bits/stdc++.h>
using namespace std;
#define ll long long int
#define str string
#define yes_or_no cout<<"YES\n":cout<<"NO\n"
#define newl cout<<endl
const int N=105;
const ll M=1e5+5;
ll dp[M];
ll val[21];
ll x,n;
bool knapsack(int m,ll sum)
{
    if(m==n)
    {

```

```

        if(sum==x)
        {
            return true;
        }
        return false;
    }
    bool x=knapsack(m+1,sum+val[m]);
    bool y=knapsack(m+1,sum-val[m]);
    return x||y;
}
int main()
{
    cin>>n>>x;
    for(int i=0;i<n;i++)
    {
        cin>>val[i];
    }
    knapsack(1,val[0])?yes_or_no;
}

```

Output:

Input	Output
5 5 1 2 3 4 5 5 2 1 2 3 4 5	YES NO

Screenshot of Accepted Bar:

224696144	Practice: Arsenic33	223339V - 14	GNU C++20 (64)	Accepted	15 ms	760 KB	2023-09-23 08:15:18	2023-09-23 08:15:18
-----------	------------------------	---------------------------------	----------------------	----------	-------	--------	------------------------	------------------------

Comments of mine:

Here I have also used recursion to check the all numbers by putting + or – sign. It is also an example of Knapsack.

Problem Name: **Reach Value (54)**

Summary of Problem:

There will be given an integer and we have to determine if we can reach that value from starting 1 by multiplying 10 or 20 any number of times.

Code:

```

#include<bits/stdc++.h>
using namespace std;
#define ll long long int
#define str string
#define yes_or_no cout<<"YES\n":cout<<"NO\n"
#define newl cout<<endl
const int N=105;
const ll M=1e5+5;
ll dp[M];
ll val[21];
ll x,n;
bool knapsack(ll sum)
{
    if(sum==n)
    {
        return true;
    }
}

```

```

    }
    else if(sum>n)
    {
        return false;
    }
    bool x=knapsack(sum*10);
    bool y=knapsack(sum*20);
    return x||y;
}
int main()
{
    int t;
    cin>>t;
    while(t--)
    {
        cin>>n;
        knapsack(1)?yes_or_no;
    }
}

```

Output:

Input	Output
5	YES
1	NO
2	YES
10	NO
25	YES
200	

Screenshot of Accepted Bar:

224696399	Practice: Arsenic33	223339W - 12	GNU C++20 (64)	Accepted	0 ms	760 KB	2023-09-23 08:18:30	2023-09-23 08:18:30
-----------	------------------------	---------------------------------	----------------------	----------	------	--------	------------------------	------------------------

Comments of mine:

Here I also used recursion to solve this problem and check that if the value could be get or not.

Problem Name: **Maximum Path Sum (55)**

Summary of Problem:

There will be given a 2D matrix and we have to determine the maximum sum by traversing right or down of the matrix.

Code:

```

#include<bits/stdc++.h>
using namespace std;
#define ll long long int
#define str string
#define newl cout<<endl
const int N=100;
const int M=100;
ll A[N+1][M+1];
int n,m;
ll dp[N+1][M+1];
bool is_inside(int x,int y)
{
    if(x<=n&& x>0&& y<=m&& y>0)
    {
        return true;
    }
}

```

```

    }
    return false;
}
ll path_sum(int x,int y)
{
    ll ans1=-1e10;
    if(x==n&& y==m)
    {
        dp[x][y]=A[x][y];
        return A[x][y];
    }
    if(dp[x][y]!=0)
    {
        return dp[x][y];
    }
    if(is_inside(x+1,y))
    {
        ans1=max(ans1,A[x][y]);
        ans1+=path_sum(x+1,y);
    }
    ll ans2=-1e10;
    if(is_inside(x,y+1))
    {
        ans2=max(ans2,A[x][y]);
        ans2+=path_sum(x,y+1);
    }
    dp[x][y]=max(ans1,ans2);
    return max(ans1,ans2);
}
int main()
{
    memset(dp,0,sizeof(dp));
    cin>>n>>m;
    for(int i=1;i<=n;i++)
    {
        for(int j=1;j<=m;j++)
        {
            cin>>A[i][j];
        }
    }
    cout<<path_sum(1,1);endl;
}

```

Output:

Input	Output
3 3 5 2 4 1 3 5 9 2 7	24

Screenshot of Accepted Bar:

226460964	Practice: Arsenic33	223339X - 14	GNU C++20 (64)	Accepted	15 ms	136 KB	2023-10-03 17:56:23	2023-10-03 17:56:23
-----------	------------------------	---------------------------------	----------------------	----------	-------	--------	------------------------	------------------------

Comments of mine:

Here I have used recursion method and starting from the element (1,1) and traversing all the possible way and get the maximum sum from it.

Problem Name: **Number of Ways (56)**

Summary of Problem:

There will be given 2 integers and we have to determine the way to get the last integers by adding with the first value 1 or 2 or 3.

Code:

```
#include<bits/stdc++.h>
using namespace std;
#define ll long long int
#define str string
#define newl cout<<endl
int n,m;
ll ways(int x)
{
    ll ans=0;
    if(x==m)
    {
        return 1;
    }
    if(x>m)
    {
        return 0;
    }
    ans+=ways(x+1);
    ans+=ways(x+2);
    ans+=ways(x+3);
    return ans;
}
int main()
{
    cin>>n>>m;
    cout<<ways(n);
}
```

Output:

Input	Output
2 5	4

Screenshot of Accepted Bar:

224703056	Practice: Arsenic33	223339Y - 13	GNU C++20 (64)	Accepted	15 ms	0 KB	2023-09-23 09:25:05	2023-09-23 09:25:05
-----------	------------------------	---------------------------------	----------------------	----------	-------	------	------------------------	------------------------

Comments of mine:

Here I have used a function name 'ways' which recursively adding 1, 2, 3 from the recursion method.

Problem Name: **Apple Division (57)**

Summary of Problem:

There will be given n integers and we have to divide it into 2 parts such a way that the difference between of the 2 arrays summation is minimum.

Code:

```
#include<bits/stdc++.h>
using namespace std;
#define ll long long int
```



```

#define str string
#define ev(x) x%2==0
#define od(x) x%2==1
#define P_B push_back
#define newl cout<<endl
int main()
{
    int n;
    cin>>n;
    ll sum=0;
    vector<ll>A(n),B;
    for(int i=0;i<n;i++)
    {
        cin>>A[i];
        sum+=A[i];
    }
    ll x=1<<n;
    for(int i=0;i<x-1;i++)
    {
        ll a=i,b=0;
        ll sum2=0;
        while(a!=0)
        {
            if(od(a))
            {
                sum2+=A[b];
            }
            b++;
            a=a>>1;
        }
        B.P_B(sum2);
    }
    ll mn=1e18;
    for(ll i:B)
    {
        ll a=sum-i;
        mn=min(mn,abs(i-a));
    }
    cout<<mn;
}

```

Output:

Input	Output
5 3 2 7 4 1	1

Screenshot of Accepted Bar:

Task:	Apple Division
Sender:	Arsenic
Submission time:	2023-10-06 18:23:55 +0300
Language:	C++20
Status:	READY
Result:	ACCEPTED

Comments of mine:

Here I have used Bit manipulation technique to generate all the subsets as n is less bigger. Then calculated the sum and determined the minimum.

Problem Name: **Permutation (58)**

Summary of Problem:

There will be given an array of integers and we have to determine all the combination of of the array.

Code:

```
#include<bits/stdc++.h>
using namespace std;
#define ll long long int
#define str string
#define newl cout<<endl

class Solution
{
public:
    vector<vector<int>> permute(vector<int>& nums)
    {
        map<int,int>mp;
        for(int i=0;i<nums.size();i++)
        {
            if(nums[i]==0)
            {
                mp[20]=nums[i];
                nums[i]=20;
            }
            else if(nums[i]<0)
            {
                mp[21+i]=nums[i];
                nums[i]=21+i;
            }
        }
        sort(nums.begin(),nums.end());
        vector<vector<int>>A;
        do
        {
            A.push_back(nums);
        }
        while(next_permutation(nums.begin(),nums.end()));
        for(int i=0;i<A.size();i++)
        {
            for(int j=0;j<A[i].size();j++)
            {
                if(A[i][j]>=20)
                {
                    A[i][j]=mp[A[i][j]];
                }
            }
        }
        return A;
    }
};

int main()
{
    Solution S;
    int n;
    cin>>n;
    vector<int>A(n);
    for(int i=0;i<n;i++)
```

```

{
    cin>>A[i];
}
vector<vector<int>>permuted=S.permute(A);
for(int i=0;i<permuted.size();i++)
{
    for(int it:permuted[i])
    {
        cout<<it<<" ";
    }
    newl;
}
}

```

Output:

Input	Output
[1,2,3]	[[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]

Screenshot of Accepted Bar:



Comments of mine:

Here I have used a built-in function name 'next_permutation' to rearrange the array and used some technique to avoid 0 and negative value for avoiding errors.

Problem Name: **Subsets (58)**

Summary of Problem:

There will be given an array and we have to generate all the subsets including empty set.

Code:

```

#include<bits/stdc++.h>
using namespace std;
#define ll long long int
#define str string
#define od(x) x%2==1
#define newl cout<<endl

```

class Solution

```

{
    public:
    vector<vector<int>> subsets(vector<int>& nums)
    {
        vector<vector<int>>A;
        int n=nums.size();
        int x=1<<n;
        for(int i=0;i<x;i++)
        {
            vector<int>help;
            int m=i,x=0;
            while(m!=0)
            {
                if(od(m))
                {

```

```

        help.push_back(nums[x]);
    }
    x++;
    m=m>>1;
}
A.push_back(help);
}
return A;
}
};
int main()
{
    Solution S;
    vector<int>A;
    int n;cin>>n;
    for(int i=0;i<n;i++)
    {
        int a;
        cin>>a;
        A.push_back(a);
    }
    vector<vector<int>>sets=S.subsets(A);
    for(int i=0;i<sets.size();i++)
    {
        for(int it:sets[i])
        {
            cout<<it<<" ";
        }
        newl;
    }
}
}

```

Output:

Input	Output
[1,2,3]	[],[1],[2],[1,2],[3],[1,3],[2,3],[1,2,3]

Screenshot of Accepted Bar:



Comments of mine:

Here I have used bit manipulation technique to generate all the subsets and every vector is inserted into a 2D vector.

Problem Name: Static Range Sum Queries (60)

Summary of Problem:

There will be given an array and we have to calculate the sum from the queries.

Code:

```

#include<bits/stdc++.h>
using namespace std;
#define ll long long int
#define str string
#define newl cout<<endl
int main()

```

```

{
    int n,q;
    cin>>n>>q;
    vector<ll>A(n),B;
    B.push_back(0);
    for(int i=0;i<n;i++)
    {
        cin>>A[i];
        B.push_back(B.back()+A[i]);
    }
    while(q--)
    {
        int a,b;
        cin>>a>>b;
        cout<<B[b]-B[a-1];
        newl;
    }
}

```

Output:

Input	Output
8 4	11
3 2 4 5 1 1 5 3	2
2 4	24
5 6	4
1 8	
3 3	

Screenshot of Accepted Bar:

Task:	Static Range Sum Queries
Sender:	Arsenic
Submission time:	2023-10-03 21:33:28 +0300
Language:	C++20
Status:	READY
Result:	ACCEPTED

Comments of mine:

Here I have used prefix sum technique to solve this problem as the time limit is too short. When come any queries I have just printed the subtraction of last and first-1;

Problem Name: Forest Queries (61)

Summary of Problem:

There will be given a 2D matrix and we have to calculate the sum of the sub 2D matrix.

Code:

```

#include<bits/stdc++.h>
using namespace std;
#define ll long long int
#define str string
#define newl cout<<endl
const int N=1000;
int A[N][N],B[N+1][N+1];
int main()
{

```

```

int n,q;
cin>>n>>q;
for(int i=0;i<n;i++)
{
    str c;
    cin>>c;
    for(int j=0;j<n;j++)
    {
        A[i][j]=(c[j]=='*')?1:0;
    }
}
for(int i=1;i<=n;i++)
{
    for(int j=1;j<=n;j++)
    {
        B[i][j]=A[i-1][j-1]+B[i-1][j]+B[i][j-1]-B[i-1][j-1];
    }
}
while(q--)
{
    int a,b,c,d;
    cin>>a>>b>>c>>d;
    cout<<B[c][d]-B[a-1][d]-B[c][b-1]+B[a-1][b-1];
    newl;
}
}

```

Output:

Input	Output
4 3 .*.. *.*. **.. **** 2 2 3 4 3 1 3 1 1 1 2 2	3 1 2

Screenshot of Accepted Bar:

Task:	Forest Queries
Sender:	Arsenic
Submission time:	2023-10-06 20:56:40 +0300
Language:	C++20
Status:	READY
Result:	ACCEPTED

Comments of mine:

Here I have used 2D prefix sum to calculate the summation and I have created an array of 0 and 1 where 1 stands for '*'.

Problem Name: **Little Girl and Maximum Sum (62)**

Summary of Problem:

There will be given an array and we have to rearrange the array such way that we can get maximum summation for total queries.

Code:

```

#include<bits/stdc++.h>
using namespace std;
#define ll long long int
#define str string
#define A_sort(x) sort(x.begin(),x.end())
#define newl cout<<endl
int main()
{
    int n,q;
    cin>>n>>q;
    vector<ll>A(n),mp(n+1,0);
    for(int i=0;i<n;i++)
    {
        cin>>A[i];
    }
    A_sort(A);
    for(int i=1;i<=q;i++)
    {
        int l,r;
        cin>>l>>r;
        mp[l-1]++;
        mp[r]--;
    }
    for(int i=1;i<n;i++)
    {
        mp[i]+=mp[i-1];
    }
    A_sort(mp);
    ll sum=0;
    for(int i=0;i<n;i++)
    {
        sum+=A[i]*mp[i+1];
    }
    cout<<sum<<endl;
}

```

Output:

Input	Output
3 3 5 3 2 1 2 2 3 1 3	25

Screenshot of Accepted Bar:

217130287	Practice: Arsenic33	276C - 13	GNU C++20 (64)	Accepted	108 ms	3144 KB	2023-08-04 17:23:02	2023-08-04 17:23:02
-----------	------------------------	---------------------------	----------------------	----------	-----------	---------	------------------------	------------------------

Comments of mine:

Here first of all I have sorted the array and then sorted the queries by doing -1 or +1 and then multiply the value with the array elements.

Problem Name: **Grey and Array (63)**

Code:

Could not understand the logic.

Problem Name: **Range XOR Queries (64)**

Summary of Problem:

There will be given an array and we have to determine xor operation from each queries.

Code:

```
#include<bits/stdc++.h>
using namespace std;
#define ll long long int
#define str string
#define newl cout<<endl
int main()
{
    int n,q;
    cin>>n>>q;
    vector<ll>A(n),B;
    B.push_back(0);
    for(int i=0;i<n;i++)
    {
        cin>>A[i];
        B.push_back(B.back()^A[i]);
    }
    while(q--)
    {
        int a,b;
        cin>>a>>b;
        cout<<(B[b]^B[a-1]);
        newl;
    }
}
```

Output:

Input	Output
8 4	3
3 2 4 5 1 1 5 3	0
2 4	6
5 6	4
1 8	
3 3	

Screenshot of Accepted Bar:

Task:	Range Xor Queries
Sender:	Arsenic
Submission time:	2023-10-03 21:46:26 +0300
Language:	C++20
Status:	READY
Result:	ACCEPTED

Comments of mine:

Here I have used Prefix XOR operation to do that. It looks like the prefix sum but change in operator..